université
PARIS-SACLAY

RAPPORT DE TP

Programmation parallèle sur GPU et FPGA

Auteur :

Gaëtan BOUCHY

Enseignant :

M. GAC

À Orsay, le 01/03/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
TP0 : Analyse des performances d'algorithmes de calculs matriciels 3
Algorithme 1 : matrix_mul_CUDA_C 3
Algorithme 2 : matrix_mul_CUDA_v0 5
Algorithme 3 : matrix_mul_CUDA_v0_bis 7
Algorithme 4 : matrix_mul_CUDA_v1 9
Algorithme 5 : matrix_mul_CUBLAS 12
Comparaison des performances de chaque algorithme 14
TP1 : Parallélisation d'algorithmes sur GPU embarqué 16
1 - Etude algorithmique (DSL Matlab) 16
1.1 Ferrari rouge 16
1.2 Ferrari jaune 18
1.3 Optimisation de code Matlab 19
2 - Exécution sur CPU (langage C) 20
2.1 Code à trou 20
2.2 Optimisation de code sur CPU 22
3. Parallélisation sur GPU (langage CUDA) 24
3.1 Puissance de calcul de la carte 24
3.2 Code à trou 25
3.3 Accélération Obtenue 28
TP2 : Réduction Sommation parallèle avec CUDA 29
Algorithme 1 : reduce_kernel 29
Algorithme 2 : reduce_kernel_optimized 31
Analyse des performances 33
TP3 : Computer vision 34
Algorithme 1 : kernel_seuillageGPU 34
Algorithme 2 : kernel_AlgorithmeSobel 37

université
PARIS-SACLAY

TP0 : Analyse des performances d'algorithmes de calculs matriciels

Algorithme 1 : matrix_mul_CUDA_C

Cet algorithme utilise le langage C ainsi que CUDA, une plateforme de parallélisation de calculs sur GPU pour accélérer les multiplications de matrices. Il utilise les événements CUDA pour mesurer le temps d'exécution du code sur GPU et effectue les opérations suivantes :

- Allocation de la mémoire sur le GPU pour les matrices A, B et C
- Transfert de la mémoire depuis le CPU vers le GPU pour les matrices A et B
- Appel de la fonction de multiplication de matrices sur GPU
- Transfert de la mémoire de la matrice C du GPU vers le CPU
- Libération de la mémoire sur le GPU

Cependant, le code de calcul matriciel est écrit en C. Il serait possible de rendre ce code plus rapide en l'écrivant en CUDA pour utiliser les capacités de calcul parallèle de la carte GPU. Le code pourrait alors être exécuté sur plusieurs cœurs de la carte GPU en même temps :

```
void computeGold(float* C, const float* A, const float* B, unsigned int hA, unsigned int wA, unsigned int wB)
{
    for (unsigned int i = 0; i < hA; ++i)
        for (unsigned int j = 0; j < wB; ++j) {
            double sum = 0;
            for (unsigned int k = 0; k < wA; ++k) {
                double a = A[i * wA + k];
                double b = B[k * wB + j];
                sum += a * b;
            }
            C[i * wB + j] = (float)sum;
        }
}
```

*Figure 1 : Code CUDA pour le calcul matriciel (A*B stocké dans C)*

Lorsque nous mesurons le temps d'exécution de cet algorithme pour différentes tailles de matrice, voici ce que nous obtenons :

```
matrix_Mul_CUDA_C

MATRICE DE TAILLE 128
Time=                                   36.149 msec
MATRICE DE TAILLE 256
Time=                                   388.194 msec
MATRICE DE TAILLE 512
Time=                                   2412.274 msec
MATRICE DE TAILLE 1024
Time=                                   30800.621 msec
```

Figure 2 : Temps d'exécution de l'algorithme matrix_mul_CUDA_C

Bien évidemment, nous remarquons que plus la taille de la matrice augmente, plus le temps d'exécution augmente également même si cela ne semble pas linéaire.

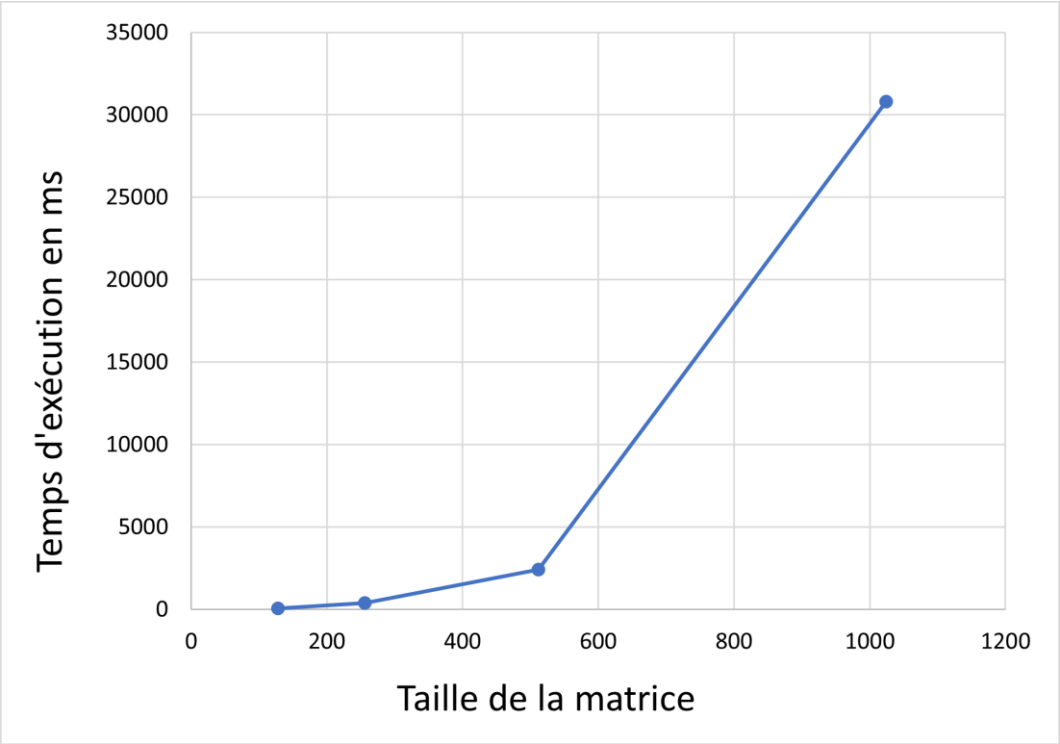


Figure 3 : Graphique du temps d'exécution de l'algorithme matrix_mul_CUDA_C en fonction de la taille de la matrice

Algorithme 2 : matrix_mul_CUDA_v0

```
__global__ void
matrixMul_v0( float* C, float* A, float* B,int matrix_size)
{
  int k;
  float C_sum;

  int i_0,j_0;
  int i,j;

  int a,b,c;

  i_0=blockIdx.x*BLOCK_SIZE;
  j_0=blockIdx.y*BLOCK_SIZE;

  i=i_0+threadIdx.x;
  j=j_0+threadIdx.y;

  a=i*matrix_size;
  b=j;
  c=j+i*matrix_size;


  for (k = 0; k < matrix_size; ++k){
    C_sum += A[a] * B[b];
    a++;
    b+=matrix_size;
  }

  C[c] = C_sum;
}
```

Figure 4 : Code CUDA pour le calcul matriciel

Ce code CUDA implémente un algorithme de multiplication de matrices en utilisant le traitement parallèle sur GPU. Voici les différentes étapes de fonctionnement de celui-ci :
- L'annotation "global" indique que cette fonction est exécutée sur le GPU. La fonction prend en entrée 4 tableaux de flottants, « **C** », « **A** », « **B** » et un entier « **matrix_size** », qui représentent les matrices de sortie, d'entrée 1 et 2, respectivement, et la taille des matrices
- Le noyau (kernel) décompose la matrice en blocs de taille **BLOCK_SIZE** et les traite de manière parallèle en utilisant les threads. Chaque bloc est identifié par les coordonnées du bloc, « **blockIdx.x** » et « **blockIdx.y** », qui sont utilisées pour déterminer la position de départ du bloc dans les matrices A et B

- Chaque thread dans un bloc est identifié par ses coordonnées « **threadIdx.x** » et « **threadIdx.y** », qui sont utilisées pour déterminer la position dans les matrices A et B où le traitement doit être effectué
- La multiplication de matrices est effectuée en multipliant chaque élément de la ligne i dans la matrice A par chaque élément de la colonne j dans la matrice B et en additionnant les produits. Le résultat est stocké dans la matrice C à la position correspondante.

Lorsque nous mesurons le temps d'exécution de cet algorithme pour différentes tailles de matrice, voici ce que nous obtenons :

```
matrix_Mul_CUDA_v0

MATRICE DE TAILLE 128
TOTAL :                         26.798023 (ms) dont 9.75% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         73.389122 (ms) dont 3.41% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         208.085114 (ms) dont 2.92% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         699.593567 (ms) dont 2.82% de gestion mémoire
```

Figure 5 : Temps d'exécution de l'algorithme Matrix_Mul_CUDA_v0

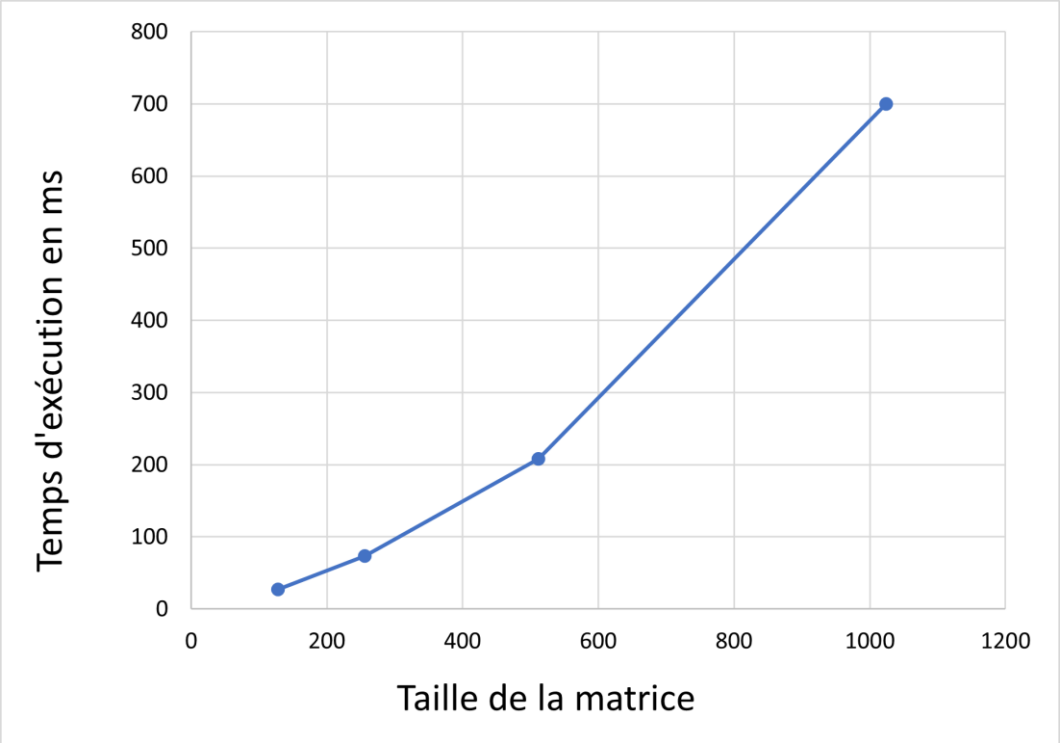


Figure 6 : Graphique du temps d'exécution de l'algorithme Matrix_Mul_CUDA_v0 en fonction de la taille de la matrice

Le partitionnement en blocs et les threads permettent d'optimiser l'utilisation de la mémoire et de la puissance de traitement du GPU pour améliorer les performances de ce code de calcul matriciel (vis-à-vis du premier algorithme).

Algorithme 3 : matrix_mul_CUDA_v0_bis

```
__global__ void
matrixMul_v0_bis( float* C, float* A, float* B,int matrix_size)
{
  int k;
  float C_sum;

  int i_0,j_0;
  int i,j;

  int a,b,c;

  j_0=blockIdx.x*BLOCK_SIZE;
  i_0=blockIdx.y*BLOCK_SIZE;

  j=j_0+threadIdx.x;
  i=i_0+threadIdx.y;

  a=i*matrix_size;
  b=j;
  c=j+i*matrix_size;


  for (k = 0; k < matrix_size; ++k){
    C_sum += A[a] * B[b];
    a++;
    b+=matrix_size;
  }

  C[c] = C_sum;

}
```

Figure 7 : Code CUDA pour le calcul matriciel

Ce code est similaire à l'algorithme précédent « **matrix_mul_CUDA_v0** ». Cependant, la seule différence est l'ordre de boucle dans laquelle les indices de bloc et de fil sont utilisés pour accéder aux matrices. Dans « **matrix_mul_CUDA_v0** », les indices de « bloc » sont utilisés pour trouver la position de départ des lignes dans la matrice C, tandis que les indices de « thread » sont utilisés pour trouver la position de départ des colonnes. Dans « **matrix_mul_CUDA_v0_bis** », les indices de « bloc » sont utilisés pour trouver la

position de départ des colonnes, tandis que les indices de « thread » sont utilisés pour trouver la position de départ des lignes.

Lorsque nous mesurons le temps d'exécution de cet algorithme pour différentes tailles de matrice, voici ce que nous obtenons :

```
matrix_Mul_CUDA_v0_bis

MATRICE DE TAILLE 128
TOTAL :                         2.020313 (ms) dont 82.85% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         4.124584 (ms) dont 35.51% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         28.853231 (ms) dont 16.93% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         189.275879 (ms) dont 9.64% de gestion mémoire
```

Figure 8 : Temps d'exécution de l'algorithme Matrix_Mul_CUDA_v0_bis

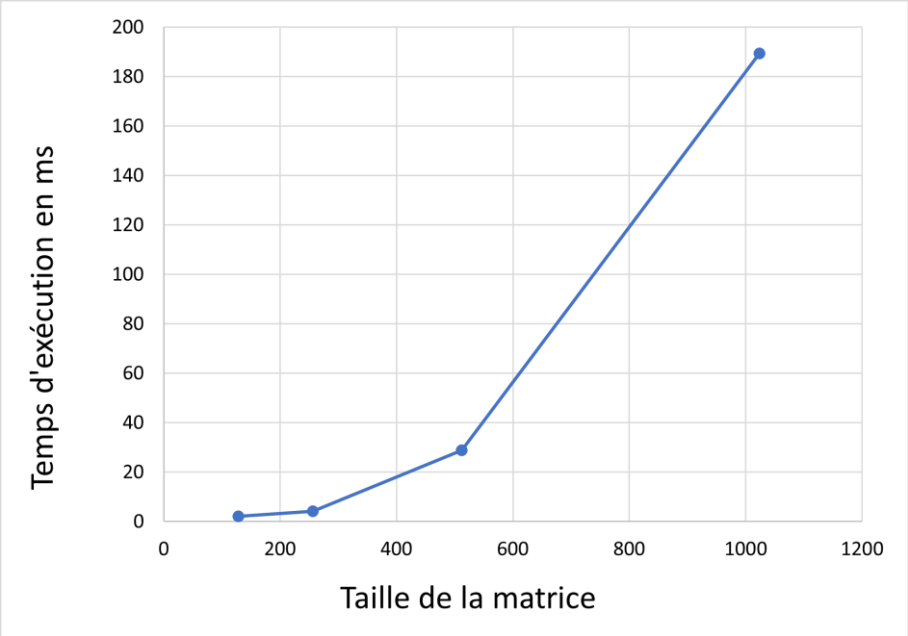


Figure 9 : Graphique du temps d'exécution de l'algorithme Matrix_Mul_CUDA_v0_bis en fonction de la taille de la matrice

Nous remarquons que les temps d'exécution de « **matrix_mul_CUDA_v0_bis** » sont bien inférieur à ceux de « **matrix_mul_CUDA_v0** ». Les raisons principales pour cela sont la gestion de la mémoire et l'utilisation des caches du processeur.

Dans « **matrix_mul_CUDA_v0_bis** », les accès à la mémoire sont enregistrés par colonnes, ce qui signifie que les données se trouvent dans la même zone de la mémoire pendant le traitement de la même colonne (d'où une utilisation élevée de gestion mémoire). Cela améliore l'utilisation des caches et réduit les temps d'accès à la mémoire.

Au contraire, dans « **matrix_mul_CUDA_v0** », les accès à la mémoire sont enregistrés par lignes, ce qui signifie que les données sont dispersées dans la mémoire pendant le traitement de la même ligne. Cela peut entraîner une surcharge de traitement pour la gestion des caches et un temps d'accès à la mémoire plus long.

Algorithme 4 : matrix_mul_CUDA_v1

```
__global__ void
matrixMul_v1( float* C, float* A, float* B, int wA, int wB)
{
    int bx = blockIdx.x;
    int by = blockIdx.y;

    int tx = threadIdx.x;
    int ty = threadIdx.y;

    int aBegin = wA * BLOCK_SIZE * by;
    int aEnd   = aBegin + wA - 1;
    int aStep  = BLOCK_SIZE;
    int bBegin = BLOCK_SIZE * bx;
    int bStep  = BLOCK_SIZE * wB;
    float Csub = 0;

    for (int a = aBegin, b = bBegin;
             a <= aEnd;
             a += aStep, b += bStep) {

        __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
        __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];

        AS(ty, tx) = A[a + wA * ty + tx];
        BS(ty, tx) = B[b + wB * ty + tx];

        __syncthreads();

        for (int k = 0; k < BLOCK_SIZE; ++k)
            Csub += AS(ty, k) * BS(k, tx);

        __syncthreads();
    }
```

Figure 10 : Code CUDA pour le calcul matriciel

Ce code utilise un modèle de mémoire partagé pour améliorer les performances. Le travail est effectué par un bloc de threads CUDA, où chaque thread est responsable de calculer un seul élément du produit matriciel.

Au début, les blocs sont indexés en utilisant « **blockIdx.x** » et « **blockIdx.y** » et chaque thread est indexé en utilisant « **threadIdx.x** » et « **threadIdx.y** ».

Ensuite, le code calcule les indices de début et de fin des sous-matrices de A et de B qui seront utilisées pour le calcul du produit matriciel. Les mémoires partagées As et Bs sont déclarées pour stocker les sous-matrices de A et de B.

Puis le code charge les sous-matrices de A et de B depuis la mémoire du dispositif vers la mémoire partagée. Le code utilise la synchronisation « **__syncthreads** » pour s'assurer que les sous-matrices sont correctement chargées avant leur utilisation.

Enfin, le code effectue la multiplication de matrices en utilisant la mémoire partagée et stocke le résultat dans « **Csub** ». Une autre synchronisation « **__syncthreads** » est utilisée pour s'assurer que la multiplication est terminée avant de charger de nouvelles sous-matrices de A et de B pour la prochaine itération. Finalement, le code écrit le produit matriciel calculé dans la mémoire du dispositif.

Lorsque nous mesurons le temps d'exécution de cet algorithme pour différentes tailles de matrice, voici ce que nous obtenons :

```
matrix_Mul_CUDA_v1

MATRICE DE TAILLE 128
TOTAL :                         3.302396 (ms) dont 93.79% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         2.754844 (ms) dont 54.77% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         15.369323 (ms) dont 30.76% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         93.045990 (ms) dont 20.24% de gestion mémoire
```

Figure 11 : Temps d'exécution de l'algorithme Matrix_Mul_CUDA_v1

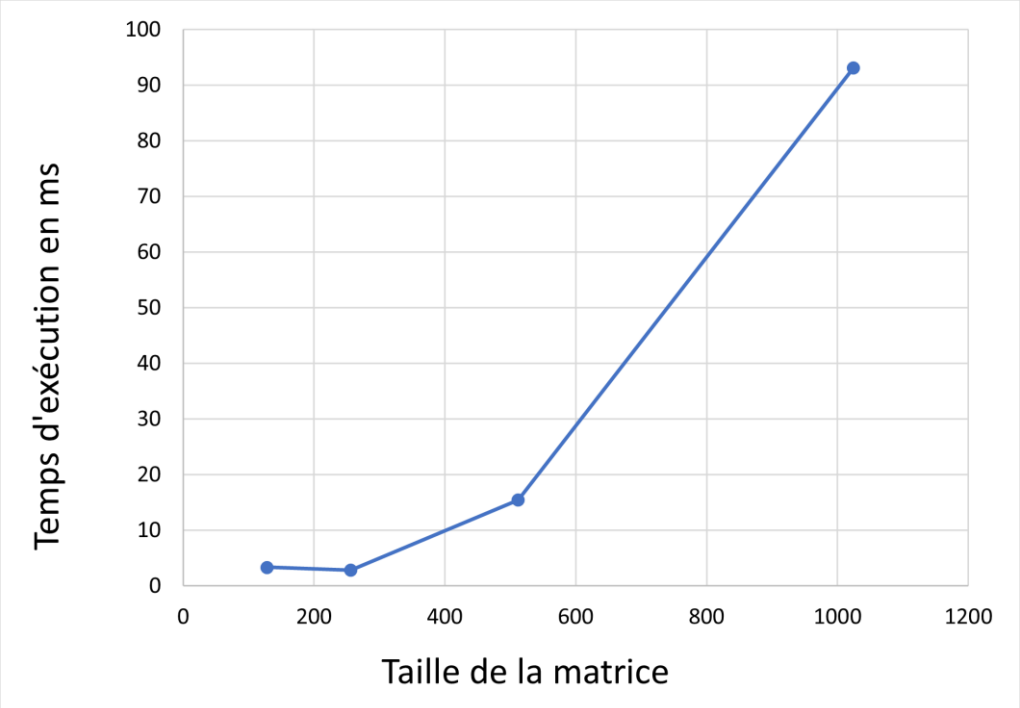


Figure 12 : Graphique du temps d'exécution de l'algorithme Matrix_Mul_CUDA_v1 en fonction de la taille de la matrice

Ce code « **Matrix_Mul_CUDA_v1** » a globalement de meilleures performances que celui de « **Matrix_Mul_CUDA_v0_bis** » car il utilise de la mémoire partagée pour stocker les sous-matrices A et B. Cela réduit les temps d'accès à la mémoire globale et améliore les performances en parallélisant les opérations.

De plus, ce code utilise une synchronisation des threads pour s'assurer que les calculs précédents sont terminés avant de charger de nouvelles sous-matrices de A et B, ce qui améliore également les performances en réduisant les dépendances entre les threads.

Algorithme 5 : matrix_mul_CUBLAS

Le programme prend en entrée une valeur N et calcule la multiplication de deux matrices carrées de dimension NxN. Les matrices sont initialisées avec des nombres aléatoires et les résultats sont stockés dans la mémoire hôte.

Le code initialise CUBLAS (bibliothèque CUBLAS) et alloue de la mémoire pour les matrices sur le GPU. Les matrices A et B sont initialisées avec des valeurs aléatoires sur la mémoire hôte et copiées sur la mémoire GPU. Ensuite, la fonction « **cublasSgemm** » de la bibliothèque CUBLAS est utilisée pour effectuer la multiplication matricielle de réels en virgule flottante en utilisant la méthode de multiplication de matrices générale (GEMM). Les résultats sont ensuite copiés de nouveau sur la mémoire hôte pour une utilisation ultérieure.

Lorsque nous mesurons le temps d'exécution de cet algorithme pour différentes tailles de matrice, voici ce que nous obtenons :

```
matrix_Mul_CUBLAS

MATRICE DE TAILLE 128
TOTAL :                         24.903803 (ms) dont 1.52% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         0.905625 (ms) dont 72.61% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         6.770989 (ms) dont 79.74% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         28.699480 (ms) dont 65.68% de gestion mémoire
```

Figure 13 : Temps d'exécution de l'algorithme Matrix_Mul_CUBLAS

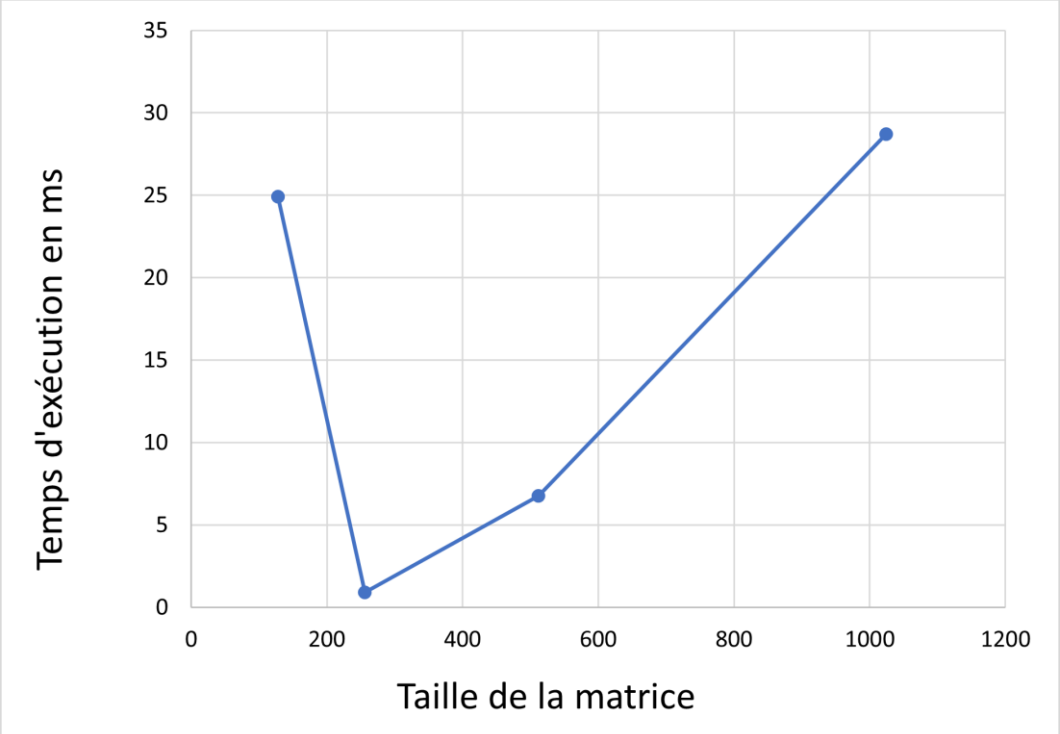


Figure 14 : Graphique du temps d'exécution de l'algorithme Matrix_Mul_CUDA_CUBLAS en fonction de la taille de la matrice

Hormis le premier résultat qui semble incohérent (peut-être dû à un problème en lien avec l'OS). Les temps d'exécution sont nettement meilleurs.

En effet, l'algorithme « **matrix_mul_CUBLAS** » utilise l'API CUBLAS de NVIDIA, qui est une bibliothèque optimisée pour les opérations de calcul sur des matrices sur des GPU NVIDIA. En utilisant CUBLAS, le code bénéficie d'optimisations matérielles et logicielles qui accélèrent les opérations de multiplication de matrices. Certaines de ces optimisations comprennent :

- Utilisation de la mémoire GPU pour les données et les résultats : CUBLAS utilise la mémoire GPU pour stocker les données matricielles, ce qui permet une accélération considérable des calculs par rapport à l'utilisation de la mémoire du CPU
- Implémentation efficace du produit matriciel : CUBLAS utilise des algorithmes optimisés pour effectuer le produit matriciel, qui peuvent inclure des algorithmes de Strassen, de cache blocking et de Tiling pour une utilisation optimale des ressources matérielles
- Exploitation des unités de calculs vectoriels (SIMD) : CUBLAS peut exploiter les unités de calcul vectorielles présentes sur les GPU pour effectuer des opérations de calcul en parallèle, ce qui accélère considérablement les opérations de multiplication de matrices
- Optimisation du niveau de granularité : CUBLAS utilise une granularité optimisée pour effectuer les opérations de multiplication de matrices, ce qui peut inclure l'optimisation du nombre de threads utilisés, du nombre de blocs de threads, et de la taille des blocs.

Ainsi, l'algorithme « **matrix_mul_CUBLAS** » est plus performant que « **matrix_mul_CUDA_v1** », qui ne fait pas appel à CUBLAS et utilise simplement des boucles pour effectuer les multiplications.

Comparaison des performances de chaque algorithme

Lorsque nous mesurons les temps d'exécution de chaque algorithme pour différentes tailles de matrice, voici ce que nous obtenons :

```
nvidia@n205-board:~/Desktop/TP_GPU/TP0_matrice$ ./matrice_exe
GPU Device 0: "Maxwell" with compute capability 5.3


matrix_Mul_CUDA_C

MATRICE DE TAILLE 128
Time=                           36.149 msec
MATRICE DE TAILLE 256
Time=                           388.194 msec
MATRICE DE TAILLE 512
Time=                           2412.274 msec
MATRICE DE TAILLE 1024
Time=                           30800.621 msec

matrix_Mul_CUDA_v0

MATRICE DE TAILLE 128
TOTAL :                         26.798023 (ms) dont 9.75% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         73.389122 (ms) dont 3.41% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         208.085114 (ms) dont 2.92% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         699.593567 (ms) dont 2.82% de gestion mémoire

matrix_Mul_CUDA_v0_bis

MATRICE DE TAILLE 128
TOTAL :                         2.020313 (ms) dont 82.85% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         4.124584 (ms) dont 35.51% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         28.853231 (ms) dont 16.93% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         189.275879 (ms) dont 9.64% de gestion mémoire

matrix_Mul_CUDA_v1

MATRICE DE TAILLE 128
TOTAL :                         3.302396 (ms) dont 93.79% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         2.754844 (ms) dont 54.77% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         15.369323 (ms) dont 30.76% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         93.045990 (ms) dont 20.24% de gestion mémoire

matrix_Mul_CUBLAS

MATRICE DE TAILLE 128
TOTAL :                         24.903803 (ms) dont 1.52% de gestion mémoire
MATRICE DE TAILLE 256
TOTAL :                         0.905625 (ms) dont 72.61% de gestion mémoire
MATRICE DE TAILLE 512
TOTAL :                         6.770989 (ms) dont 79.74% de gestion mémoire
MATRICE DE TAILLE 1024
TOTAL :                         28.699480 (ms) dont 65.68% de gestion mémoire
nvidia@n205-board:~/Desktop/TP_GPU/TP0_matrice$
```

Figure 15 : Différents temps d'exécution des cinq algorithmes

Puis si nous réalisons la représentation graphique de ces temps d'exécution pour des tailles de matrice différentes, voici ce que nous obtenons :

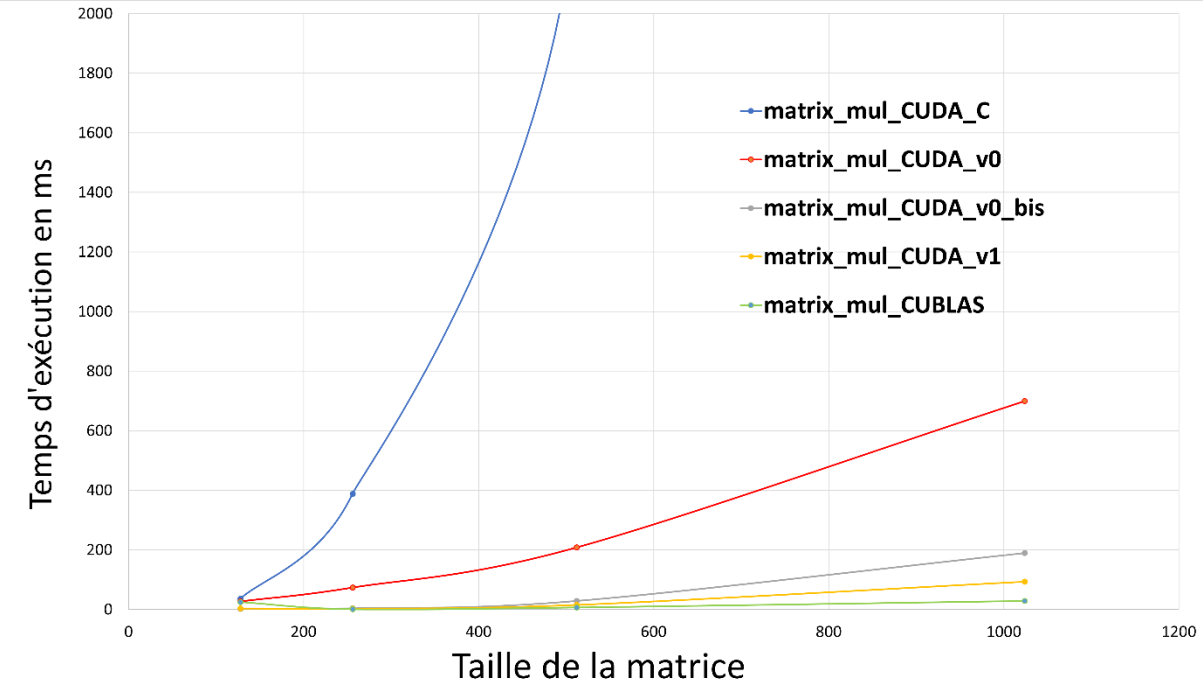


Figure 16 : Graphique du temps d'exécution de chaque algorithme en fonction de la taille de la matrice

Ces valeurs montrent les temps d'exécution des différentes versions d'une opération de multiplication de matrices pour des tailles de matrices différentes. Les implémentations incluent « **matrix_mul_CUDA_C** », « **matrix_mul_CUDA_v0** », « **matrix_mul_CUDA_v0_bis** », « **matrix_mul_CUDA_v1** » et « **matrix_mul_CUBLAS** ».

Il est clair que « **matrix_mul_CUBLAS** » est beaucoup plus performant que les autres implémentations, en particulier pour les tailles de matrices plus grandes. « **matrix_mul_CUDA_C** » est également assez rapide pour les petites tailles de matrices, mais son temps d'exécution augmente considérablement avec la taille de la matrice. Les autres implémentations ont également un temps d'exécution plus long que « **matrix_mul_CUBLAS** » pour les tailles de matrices plus grandes, mais restent généralement inférieures à « **matrix_mul_CUDA_C** ».

université PARIS-SACLAY

TP1 : Parallélisation d'algorithmes sur GPU embarqué

1 - Etude algorithmique (DSL Matlab)

1.1 Ferrari rouge

Nous cherchons à seuiller l'image « ferrari.jpg » suivante en fonction de η_R, le niveau de rouge de chaque pixel de l'image :



Figure 17 : Image « Ferrari.jpg » d'origine

Pour cela, nous allons utiliser les deux formules suivantes :

$$I_s(xn, yn) = \begin{cases} I(xn, yn) \text{ si } \eta_R > 0.7 \\ 0 \text{ sinon} \end{cases}$$

$$\eta_R = \frac{I_R}{\sqrt{I_R^2 + I_G^2 + I_B^2}}$$

Figure 18 : Formules afin de seuiller en fonction du niveau de rouge

Notre objectif est donc de parcourir chaque pixel de notre image, puis de calculer le niveau de rouge de ce pixel via la formule $\eta_R = \frac{I_R}{\sqrt{I_R^2+I_G^2+I_B^2}}$. En fonction de la valeur de ce résultat, nous allons soit conserver le pixel tel quel si $\eta_R > 0.7$ ou le rendre noir si $\eta_R \leq 0.7$, car un pixel avec comme couleur RGB valant (0,0,0) équivaut à un pixel noir.

Ainsi nous obtenons ce code :

```
imaSeuil = ima;

for i = 1:taille(1) % Parcours de la longueur de l'image
    for j = 1:taille(2) % Parcours de la largeur de l'image
        n = ima(i,j,1)/(sqrt(ima(i,j,1).^2+ima(i,j,2).^2+ima(i,j,3).^2)); %calcul niv de rouge
            if n <= 0.7
               imaSeuil(i,j,:) = 0; % Mise à 0 des niveaux R/G/B
            end
    end
end
```

Figure 19 : Code qui filtre en fonction du niveau de rouge et modifie les pixels si le niveau de rouge est inférieur ou égal à 0.7

Pour information, sur notre machine personnelle, ce code se réalise en environ 0.9 sec et donne comme résultat sur l'image « ferrari.jpg » :



Figure 20 : Image « ferrari.jpg » seuillée en fonction du niveau de rouge

1.2 Ferrari jaune

Nous souhaitons maintenant garder le même procédé. Cependant, si $\eta_R > 0.7$, il faut changer les pixels rouges en pixel jaune :

```
for i = 1:taille(1) % Parcours de la longueur de l'image
    for j = 1:taille(2) % Parcours de la largeur de l'image
        n = ima(i,j,1)/(sqrt(ima(i,j,1).^2+ima(i,j,2).^2+ima(i,j,3).^2)); %calcul niv de rouge
            if n > 0.7
               ima(i,j,2) = ima(i,j,1)+ima(i,j,2); %Jaune = Rouge + Vert
            end
    end
end

ima_out=ima;
```

Figure 21 : Code qui filtre en fonction du niveau de rouge et modifie les pixels rouges en pixel jaune si le niveau de rouge est supérieur à 0.7

Nous avons simplement eu besoin de l'information comme quoi le jaune est composé des couleurs rouges « **ima(i,j,1)** » et vertes « **ima(i,j,2)** » d'où notre ligne « **ima(ij,2) = ima(i,j,1) + ima(i,j,2)** »

Pour information, sur notre machine personnelle, ce code se réalise en environ 0.042 sec et donne comme résultat sur l'image « ferrari.jpg » :



Figure 22 : Image « ferrari.jpg » seuillée en fonction du niveau de rouge

1.3 Optimisation de code Matlab

Comme nous avons vu dans les codes précédents, nous réalisons deux boucles for imbriquées afin de parcourir la longueur et la largeur de l'image. Nous pouvons supprimer ces deux boucles en réalisant un « masque » avec comme condition : $\eta_R > 0.7$. Nous reprenons ensuite les lignes de code correspond à 1.2 Ferrari jaune.

```
ima_out2 = ima2; % Récupère l'image initiale
niveauRouge = ima_r./(sqrt(ima_r.^2+ima_g.^2+ima_b.^2)); % Calcul du niveau de rouge
masque = niveauRouge > 0.7; % Création d'un masque, remplace le if
ima_g(masque)=ima_g(masque)+ima_r(masque); % Jaune = rouge + vert
ima_out2(:,:,2)=ima_g; % On récupère le résultat
```

Figure 23 : Code qui filtre en fonction du niveau de rouge et modifie les pixels rouges en pixel jaune si le niveau de rouge est supérieur à 0.7

Pour information, sur notre machine personnelle, ce code se réalise en environ 0.014 sec et donne comme résultat sur l'image « ferrari.jpg » :



Figure 24 : Image « ferrari.jpg » seuillée en fonction du niveau de rouge

Entre la version non optimisée, il y a un écart de 300% ($\text{écart} = \frac{0.042\ \text{sec}}{0.014\ sec} = 3$) afin d'obtenir une image filtrée en fonction du niveau de rouge et qui modifie les pixels rouges en pixel jaune si le niveau de rouge est supérieur à 0.7 :

```
Elapsed time is 0.042115 seconds.
Elapsed time is 0.013975 seconds.
```

Figure 25 : Différence de temps entre les versions optimisées et non optimisées

2 - Exécution sur CPU (langage C)

2.1 Code à trou

Nous avons voulu reproduire le code qui permet d'obtenir la « 1.1 Ferrari rouge » c'est-à-dire que nous allons conserver le pixel tel quel si $\eta_R > 0.7$ ou le rendre noir si $\eta_R \leq 0.7$. Voici donc notre code pour réaliser cela :

```c
void seuillage_C(float image_out[][SIZE_J][SIZE_I], float image_in[][SIZE_J][SIZE_I])
{
    float n; // Niveau de rouge
    for(int j = 0; j < SIZE_J; j++) // Boucle de la taille de la longueur
    {
        for(int i = 0; i < SIZE_I; i++) // Boucle de la taille de la largeur
        {
            // Calcul du niveau de rouge d'apres la formule
            n = image_in[0][j][i]/sqrt((image_in[0][j][i]*image_in[0][j][i])+(image_in[1][j][i]*image_in[1][j][i])+(image_in[2][j][i]*image_in[2][j][i]));
            if(n > 0.7) // Si niveau de rouge > 0.7
            {
                image_out[0][j][i]=image_in[0][j][i]; // Alors le pixel de l'image en sortie est le meme que celui de l'image d'entree
                image_out[1][j][i]=image_in[1][j][i];
                image_out[2][j][i]=image_in[2][j][i];
            }
            else
            {
                image_out[0][j][i]= 0; // Sinon le pixel devient noir
                image_out[1][j][i]= 0; // Sinon le pixel devient noir
                image_out[2][j][i]= 0; // Sinon le pixel devient noir
            }
        }
    }
}
```

Figure 26 : Code qui filtre en fonction du niveau de rouge et modifie les pixels si le niveau de rouge est inférieur ou égal à 0.7

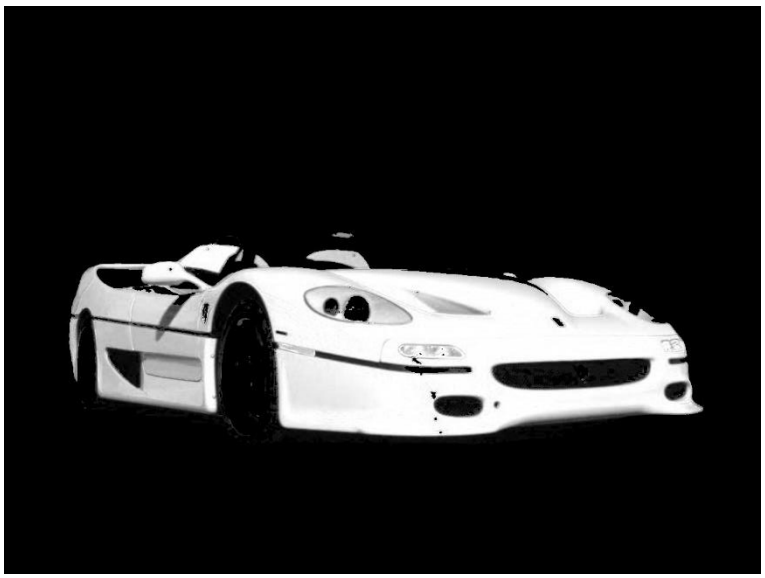


Figure 27 : Résultat du code qui filtre en fonction du niveau de rouge et modifie les pixels si le niveau de rouge est inférieur ou égal à 0.7

Initialement la Ferrari était blanche et se trouvait dans le même décor que vu précédemment. Nous pouvons voir que tous les pixels du décor ont été modifiés en pixel noir car le niveau de rouge était inférieur ou égal à 0.7. Le reste a été conservé tel quel.

Sur Matlab, comme montré dans la partie 1.1 Ferrari rouge, ce code se réalise en environ 0.7 seconde soit 700 ms :

```
Elapsed time is 0.685052 seconds.
```

Figure 28 : Temps d'exécution sur Matlab

Tandis que sur CPU, cela se réalise en seulement 120 ms soit une amélioration d'un rapport 5.8 ($\text{écart} = \frac{700\ \text{ms}}{120\ ms} = 5.8$) dans le cas où nous parcourons lignes par lignes de l'image.

2.2 Optimisation de code sur CPU

```
void seuillage_C(float image_out[][SIZE_J][SIZE_I], float image_in[][SIZE_J][SIZE_I])
{
    float n; // Niveau de rouge
    for(int j = 0; j < SIZE_J; j++) // Boucle de la taille de la longueur
    {
        for(int i = 0; i < SIZE_I; i++) // Boucle de la taille de la largeur
        {
            // Calcul du niveau de rouge d'apres la formule
            n = image_in[0][j][i]/sqrt((image_in[0][j][i]*image_in[0][j][i])+(image_in[1][j][i]*image_in[1][j][i])+(image_in[2][j][i]*image_in[2][j][i]));
            if(n > 0.7) // Si niveau de rouge > 0.7
            {
                image_out[0][j][i]=image_in[0][j][i]; // Alors le pixel de l'image en sortie est le meme que celui de l'image d'entree
                image_out[1][j][i]=image_in[1][j][i];
                image_out[2][j][i]=image_in[2][j][i];
            }
            else
            {
                image_out[0][j][i]= 0; // Sinon le pixel devient noir
                image_out[1][j][i]= 0; // Sinon le pixel devient noir
                image_out[2][j][i]= 0; // Sinon le pixel devient noir
            }
        }
    }
}
```

Figure 29 : Code qui parcourt les lignes puis les colonnes

```
void seuillage_C(float image_out[][SIZE_J][SIZE_I], float image_in[][SIZE_J][SIZE_I])
{
    float n; // Niveau de rouge
    for(int j = 0; j < SIZE_I; j++) // Boucle de la taille de la largeur
    {
        for(int i = 0; i < SIZE_J; i++) // Boucle de la taille de la longueur
        {
            // Calcul du niveau de rouge d'apres la formule
            n = image_in[0][j][i]/sqrt((image_in[0][j][i]*image_in[0][j][i])+(image_in[1][j][i]*image_in[1][j][i])+(image_in[2][j][i]*image_in[2][j][i]));
            if(n > 0.7) // Si niveau de rouge > 0.7
            {
                image_out[0][j][i]=image_in[0][j][i]; // Alors le pixel de l'image en sortie est le meme que celui de l'image d'entree
                image_out[1][j][i]=image_in[1][j][i];
                image_out[2][j][i]=image_in[2][j][i];
            }
            else
            {
                image_out[0][j][i]= 0; // Sinon le pixel devient noir
                image_out[1][j][i]= 0; // Sinon le pixel devient noir
                image_out[2][j][i]= 0; // Sinon le pixel devient noir
            }
        }
    }
}
```

Figure 30 : Code qui parcourt les colonnes puis les lignes

Lors de l'exécution du premier code, cela prenait environ 120 ms comme nous l'avons vu dans la partie précédente. Lors de l'exécution du deuxième code qui parcourt les colonnes puis les lignes, cela prenait environ 300 ms soit un écart d'un rapport 2.5 ($\text{écart} = \frac{300\ \text{ms}}{120\ ms} = 2.5$).

Le temps d'exécution peut être influencé par la manière dont les données sont stockées dans la mémoire cache de l'ordinateur. Les algorithmes qui accèdent aux données de manière séquentielle sont plus rapides car les données sont souvent contiguës dans la mémoire cache. Par conséquent, en bouclant d'abord sur les lignes puis sur les colonnes, le code 1 accède aux données de manière plus cohérente, ce qui peut réduire le nombre de fois où les données doivent être chargées à partir de la mémoire de masse (cela évite donc les « cache miss »), ce qui peut entraîner un temps d'exécution plus rapide.

En revanche, en bouclant d'abord sur les colonnes puis sur les lignes, le code peut accéder aux données de manière plus aléatoire, ce qui peut entraîner un temps d'exécution

plus lent. Cependant, cela dépend également de nombreux autres facteurs tels que la configuration du système, la taille de la mémoire cache, etc.

3. Parallélisation sur GPU (langage CUDA)

3.1 Puissance de calcul de la carte

La carte GPU Jetson Nano possède de nombreuses caractéristiques dont les principales sont :

- CPU : ARM Cortex-A57 4 cœurs à 1,43 GHZ
- GPU : NVIDIA Maxwell avec 128 cœurs CUDA à 921MHz. Performance des shaders jusqu'à 512 GFLOPS (FP16). Les shaders sont des programmes informatiques qui permettent aux processeurs graphiques de créer des images 3D en définissant la couleur, l'éclairage, les textures et autres aspects visuels des objets dans une scène.
- Mémoire RAM : 4 Go LPDDR4
- De nombreux ports : Ethernet, HDMI, 4xUSB 3.0, USB 2.0, M.2 Key E

3.2 Code à trou

3.2.1 La fonction main

L'objectif est de remplir les différentes parties du programme **runTest().** Tout d'abord, nous avons commencé par remplir les étapes d'allocation mémoire :

```
error = cudaMalloc(&d_image_in, mem_size);
error = cudaMalloc(&d_image_out, mem_size);
```

Figure 31 : Code CUDA pour l'allocation mémoire

Ces lignes de code sont utilisées pour allouer de la mémoire sur la carte GPU pour deux matrices : « **d_image_in** » et « **d_image_out** ».

La fonction « **cudaMalloc** » est utilisée pour allouer de la mémoire sur la carte GPU et prend en entrée deux arguments : l'adresse de la variable (« **&d_image_in** » et « **&d_image_out** ») et la taille de la mémoire à allouer en octets (« **mem_size** »).

L'appel à « **cudaMalloc** » retourne un code d'erreur dans la variable « error », ce qui permet de vérifier si l'allocation a réussi ou non. Si le code d'erreur est égal à zéro, cela signifie que l'allocation a réussi et que les matrices peuvent être utilisées pour stocker des données sur la carte GPU.

La seconde étape consiste à s'occuper des transferts mémoire entre le PC hôte et la carte GPU :

```
error = cudaMemcpy(d_image_in, h_image_in, mem_size, cudaMemcpyHostToDevice);
```

Figure 32 : Code CUDA pour les transferts mémoire entre le PC hôte et la carte GPU

La fonction « **cudaMemcpy** » est utilisée pour copier des données d'un emplacement mémoire à un autre et prend en entrée quatre arguments :
- La destination « **d_image_in** », où les données seront copiées sur la carte GPU
- La source « **h_image_in** », d'où les données seront copiées depuis le CPU
- La taille de la mémoire à copier en octets « **mem_size** »
- Le type de copie « **cudaMemcpyHostToDevice** » qui spécifie que les données seront copiées depuis le CPU vers la carte GPU

L'appel à « **cudaMemcpy** » retourne un code d'erreur dans la variable « **error** », ce qui permet de vérifier si la copie a réussi ou non. Si le code d'erreur est égal à zéro, cela signifie que la copie a réussi et que les données sont disponibles sur la carte GPU.

La troisième partie à remplir concerne le découpage en threads :

```
dim3 threads(16,16);
dim3 gridDim(SIZE_I/16, SIZE_J/16);
```

Figure 33 : Code CUDA pour le découpage en threads

Ces deux lignes de code définissent la configuration de la grille et des blocs. « **dim3 threads(16,16)** » définit la taille des blocs à 16 threads dans la première dimension et 16 threads dans la seconde dimension. Cela signifie que chaque bloc aura 16x16 threads.
« **dim3 gridDim(SIZE_I/16, SIZE_J/16)** » définit la grille de blocs, qui détermine le nombre de blocs qui seront exécutés par le kernel. La grille aura (« **SIZE_I/16** », « **SIZE_J/16** ») blocs, où « **SIZE_I** » et « **SIZE_J** » représentent la largeur et la longueur de l'image.

Cela permet de déterminer le nombre total de threads qui seront exécutés par le kernel en multipliant la taille des blocs par le nombre de blocs dans la grille.

Finalement la dernière partie qui consiste au lancement du kernel ainsi qu'à la copie des données du GPU au CPU :

```
seuillage_kernel<<< gridDim, threads >>>((float (*)[SIZE_J][SIZE_I])d_image_in, (float (*)[SIZE_J][SIZE_I])d_image_out);
error = cudaMemcpy(h_image_out_GPU, d_image_out, mem_size, cudaMemcpyDeviceToHost);
```

Figure 34 : Code de lancement du kernel et de copies des données

La première ligne est l'appel du kernel CUDA « **seuillage_kernel** ». Le kernel est une fonction qui sera exécutée sur le GPU en parallèle.
Le format « <<< **gridDim, threads >>>** » spécifie la configuration du lancement du kernel :
- « **gridDim** » est un objet de type « **dim3** » qui définit le nombre de blocs de threads qui seront lancés
- « **threads** » est un objet de type « **dim3** » qui définit le nombre de threads dans chaque bloc
- « **(float (*)[SIZE_J][SIZE_I])** » est utilisé pour convertir les tableaux en pointeurs

La seconde ligne copie les données du tableau « **d_image_out** » du GPU vers « **h_image_out_GPU** » sur le CPU. La fonction « **cudaMemcpy** » est utilisée pour effectuer la copie. Le dernier argument de cudaMemcpy, « **cudaMemcpyDeviceToHost** » spécifie la direction de la copie (du GPU vers le CPU).

3.2.2 Le kernel « seuillage_kernel »

En s'inspirant du code réalisé dans la partie 2.1 Code à trou. Il est facilement possible de faire un code similaire mais pour en CUDA :

```
__global__ void seuillage_kernel(float d_image_in[][SIZE_J][SIZE_I],float d_image_out[][SIZE_J][SIZE_I])
{
  int j = blockIdx.x * blockDim.x + threadIdx.x;
  int i = blockIdx.y * blockDim.y + threadIdx.y;

  if (j < SIZE_J && i < SIZE_I)
  {
      float n = d_image_in[0][j][i]/sqrt((d_image_in[0][j][i]*d_image_in[0][j][i])+(d_image_in[1][j][i]*d_image_in[1][j][i])+(d_image_in[2][j][i]*d_image_in[2][j][i]));
      if(n > 0.7)
      {
          d_image_out[0][j][i]=d_image_in[0][j][i];
          d_image_out[1][j][i]=d_image_in[1][j][i];
          d_image_out[2][j][i]=d_image_in[2][j][i];
      }
      else
      {
          d_image_out[0][j][i]= 0;
          d_image_out[1][j][i]= 0;
          d_image_out[2][j][i]= 0;
      }
    }
}
```

Figure 35 : Code qui filtre en fonction du niveau de rouge et modifie les pixels si le niveau de rouge est inférieur ou égal à 0.7

Ce code prend en entrée deux tableaux de flottants de dimensions **[SIZE_J][SIZE_I]**, nommés « **d_image_in** » et « **d_image_out** ». Les tableaux sont manipulés par des threads CUDA qui exécutent la même fonction en parallèle.

La fonction calcule un niveau de rouge pour chaque pixel de l'image en entrée en utilisant une formule donnée, puis vérifie si ce niveau de rouge est supérieur à 0.7. Si c'est le cas, les valeurs RGB du pixel sont copiées dans l'image en sortie. Sinon, les valeurs RGB du pixel de l'image en sortie sont mises à zéro.

3.3 Accélération Obtenue

Pour calculer le facteur d'accélération entre ces deux GPUs, nous pouvons utiliser la formule suivante :
Facteur d'accélération = (Nombre de cœurs CUDA de la carte A100 x Fréquence de base de la carte A100) / (Nombre de cœurs CUDA de la carte NVIDIA Maxwell x Fréquence de base de la carte NVIDIA Maxwell)

Ainsi, en utilisant les données fournies, nous obtenons concernant la carte A100 ainsi que la carte NVIDIA Maxwell :
Facteur d'accélération = (6192 x 1.41 GHz) / (128 x 921 MHz) = 74.06

Cela signifie que la carte A100 a un facteur d'accélération d'environ 74.06 fois par rapport à la carte NVIDIA Maxwell pour les calculs qui utilisent les cœurs CUDA. Cependant, il est important de noter que la performance réelle dépendra également de nombreux autres facteurs, tels que la quantité de mémoire de la carte, la bande passante mémoire, la nature des calculs effectués, …

TP2 : Réduction Sommation parallèle avec CUDA

Algorithme 1 : reduce_kernel

Le code que nous devions compléter est une implémentation de l'algorithme de réduction sommation d'un grand tableau d'entiers en utilisant le langage CUDA. La réduction est effectuée en parallèle en utilisant plusieurs threads. Plus précisément, chaque thread somme un sous-ensemble des éléments du vecteur. Le résultat de chaque sous-somme est stocké dans une mémoire partagée entre les threads du même bloc et la réduction finale est effectuée à l'aide d'un arbre de sommation dans cette mémoire partagée. Le résultat final est stocké dans un tableau de sortie, où chaque élément correspond à la somme des éléments d'un bloc.

Cela pourrait se résumer en trois étapes simples :
- Chaque thread calcule sa somme partielle
- Chaque bloc de threads calcule sa somme partielle
- Un bloc calcule la somme finale

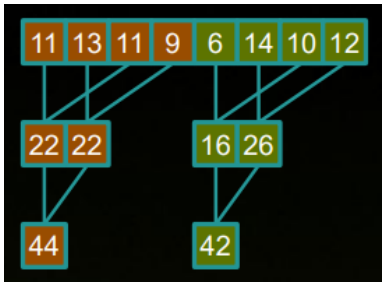


Figure 36 : Représentation graphique des étapes 1 et 2

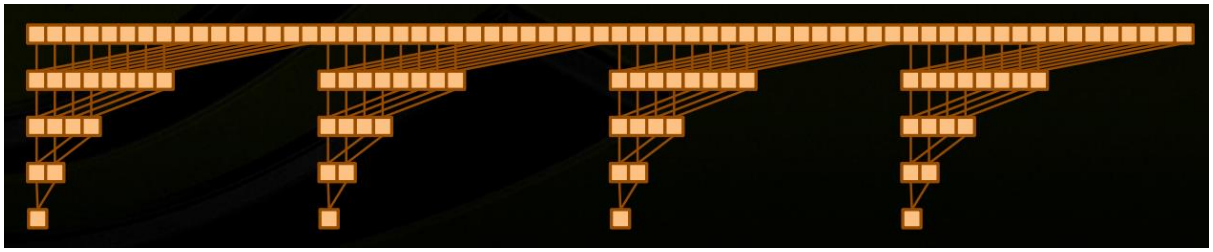


Figure 37 : Représentation graphique des étapes 1 et 2 avec plus de threads et de blocs

Finalement, la dernière étape dans la Figure 36 : Représentation graphique des étapes 1 et 2consiste à sommer les résultats obtenus par les deux blocs donc l'un des deux blocs va effectuer l'opérateur 44 + 42 = 86.

Voici le code CUDA complété pour l'algorithme de réduction-sommation détaillé précédemment :

```
__global__ void reduce_kernel( int n, const int *in_buffer, int *out_buffer, const int2 *block_ranges )
{
  extern __shared__ int s_mem[];
  int2 range = block_ranges[blockIdx.x];

  int my_sum = 0;
  for (int i = range.x + threadIdx.x; i < range.y; i += blockDim.x) {
    my_sum += in_buffer[i];
  }

  s_mem[threadIdx.x] = my_sum;
  __syncthreads();

  for (int i = blockDim.x/2; i > 0; i /= 2) {
    if (threadIdx.x < i) {
      s_mem[threadIdx.x] += s_mem[threadIdx.x + i];
    }
    __syncthreads();
  }

  if( threadIdx.x == 0 )
    out_buffer[blockIdx.x] = s_mem[0];
}
```

Figure 38 : Code CUDA pour la réduction-sommation

Voici le détail des lignes importantes concernant ce code :
- « **extern __shared__ int s_mem[]; »** : Déclaration d'un tableau s_mem de mémoire partagée qui sera utilisé pour stocker les sous-sommations de chaque thread
- « **int2 range = block_ranges[blockIdx.x];** » : Récupération de la plage d'indices du vecteur sur laquelle le bloc courant doit travailler
- « **int my_sum = 0; for (int i = range.x + threadIdx.x; i < range.y; i += blockDim.x) { my_sum += in_buffer[i]; }** » : Sommation des éléments du vecteur dans la plage range défini précédemment à l'aide d'une boucle for qui itère sur les indices correspondant à chaque thread
- « **s_mem[threadIdx.x] = my_sum;** » : Stockage de la sous-somme calculée par le thread courant dans la mémoire partagée
- « **__syncthreads();** » : Synchronisation de tous les threads du bloc pour attendre que tous aient fini d'écrire leur sous-somme dans la mémoire partagée
- **for (int i = blockDim.x/2; i > 0; i /= 2) { if (threadIdx.x < i) { s_mem[threadIdx.x] += s_mem[threadIdx.x + i]; } __syncthreads(); }** » : Réduction des sous-sommations stockées dans la mémoire partagée à l'aide d'un arbre de sommation. La synchronisation de tous les threads est requise à chaque itération pour garantir que les valeurs modifiées sont disponibles pour les autres threads
- « **if (threadIdx.x == 0) out_buffer[blockIdx.x] = s_mem[0];** »: Stockage de la somme finale calculée par le bloc dans le tableau de sortie. Cette opération est effectuée par le thread d'indice 0 de chaque bloc

Algorithme 2 : reduce_kernel_optimized

Ce deuxième code est donc là encore une implémentation d'un algorithme de réduction pour calculer la somme de tous les éléments d'un tableau en utilisant la parallélisation sur GPU.

De plus, ce code est une amélioration du code vu dans la partie précédente. Les grandes étapes de ce nouveau code sont les suivantes :

- Définition des constantes et des variables locales nécessaires pour l'exécution de l'algorithme
- Chaque thread calcule la somme partielle des éléments du tableau qui lui sont assignés
- Chaque thread copie sa somme partielle dans la mémoire partagée allouée
- Les sommes partielles dans chaque warp sont sommées entre elles pour obtenir la somme totale du warp
- Le premier thread de chaque warp stocke le résultat de la somme dans la mémoire partagée
- Le premier thread du premier warp somme les résultats de chaque warp pour obtenir la somme totale de ce bloc
- Le premier thread de chaque bloc stocke la somme totale du bloc dans le tableau de sortie

Cela pourrait se résumer en ces trois étapes simples :

- Chaque bloc calcule sa somme partielle
- Un bloc scanne les sommes des blocs (scans globaux)
- Chaque bloc analyse ses éléments et utilise son scan global

Voici le code CUDA complété pour l'algorithme optimisé de réduction-sommation détaillé précédemment :

```cuda
#define WARP_SIZE 32

template< int BLOCK_DIM >
__global__ void reduce_kernel_optimized( int n, const int *in_buffer, int *out_buffer, const int2 *__restrict block_ranges )
{
  const int NUM_WARPS = BLOCK_DIM / WARP_SIZE;
  __shared__ volatile int s_mem[BLOCK_DIM];
  int2 range = block_ranges[blockIdx.x];
  const int warp_id = threadIdx.x / WARP_SIZE;
  const int lane_id = threadIdx.x % WARP_SIZE;

  int my_sum = 0;
  for (int i = range.x + threadIdx.x; i < range.y; i += BLOCK_DIM)
    my_sum += in_buffer[i];

  s_mem[threadIdx.x] = my_sum;

  for (int i = WARP_SIZE / 2; i > 0; i /= 2) {
    if (lane_id < i) {
      s_mem[threadIdx.x] += s_mem[threadIdx.x + i];
    }
    __syncthreads();
  }

  if( lane_id == 0 )
    s_mem[warp_id] = s_mem[threadIdx.x];
  __syncthreads();

  if( warp_id == 0 )
  {
    my_sum = s_mem[lane_id];

    for (int i = 1; i < NUM_WARPS; ++i)
      my_sum += s_mem[i * WARP_SIZE];
  }

  if( threadIdx.x == 0 )
    out_buffer[blockIdx.x] = my_sum;
}
```

Figure 39 : Code CUDA optimisé pour la réduction-sommation

Voici le détail des parties importantes concernant ce code :
- Définition de la taille d'un warp « **WARP_SIZE** » à 32.
- Allocation d'une mémoire partagée à l'intérieur du bloc « **s_mem** » pour stocker les résultats intermédiaires
- Récupération de la plage d'indices à traiter par le bloc en cours
- Attribution d'un ID de warp et d'un ID de lane à chaque thread, en utilisant « **threadIdx.x** »
- Chaque thread calcule la somme de ses éléments dans la plage d'indices qui lui est attribuée
- Copie de la somme calculée par chaque thread dans la mémoire partagée du bloc
- Les threads exécutent une réduction en utilisant l'algorithme de la somme de réduction parallèle. Chaque warp calcule la somme de ses éléments en faisant des additions successives et stocke le résultat dans la mémoire partagée
- Chacun des premiers threads des warp stocke le résultat de la somme de son warp dans la mémoire partagée
- Le premier warp lit les résultats stockés dans la mémoire partagée et somme ces résultats pour obtenir la somme totale pour le bloc
- Le premier thread stocke la somme totale pour le bloc dans le tableau de sortie

Analyse des performances

Lorsque nous mesurons le temps d'exécution de ces deux algorithmes pour un vecteur de taille assez conséquente, voici ce que nous obtenons :

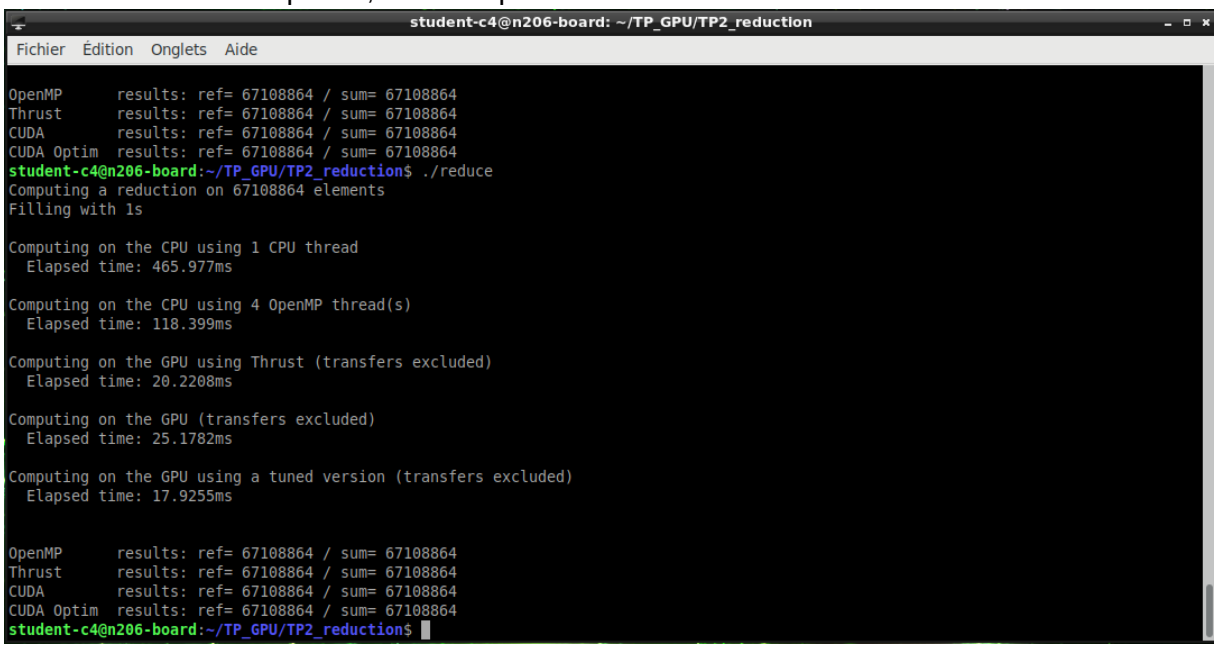


Figure 40 : Temps d'exécution des programmes reduce_kernel et reduce_kernel_optimized

Avec la version non optimisée, nous obtenons un temps d'exécution de l'ordre de 25 ms alors qu'avec la version optimisée, nous obtenons un temps d'exécution de seulement 18 ms. Cela correspond à une réduction du temps d'exécution ($\frac{25}{18} = 1.39$) d'environ 39%.

C'est pourquoi nous pouvons dire que le deuxième code est une version optimisée du premier. Voici quelques-uns de ses points forts qui expliquent cette différence :
- Utilisation des registres : chaque thread stocke sa somme partielle dans un registre plutôt que dans la mémoire partagée, ce qui réduit le nombre d'accès à la mémoire partagée
- Utilisation de la mémoire partagée volatile : la mémoire partagée est déclarée comme volatile, ce qui indique au compilateur que cette mémoire est susceptible d'être modifiée à tout moment par un autre thread. Cela garantit que les valeurs stockées dans la mémoire partagée sont toujours à jour et ne sont pas lues à partir d'une copie en cache obsolète
- Utilisation des warps : plutôt que de sommer les valeurs de chaque thread individuellement, les threads sont organisés en groupes de 32 (warps), et les sommes partielles sont sommées à l'intérieur de chaque warp, ce qui permet d'optimiser l'utilisation des unités de calcul SIMD du GPU
- Utilisation de l'instruction « __**restrict** » : l'utilisation de l'attribut « __**restrict** » indique au compilateur que les pointeurs d'entrée et de sortie ne se chevauchent pas, ce qui lui permet d'optimiser la génération de code.

Dans l'ensemble, le deuxième code est plus optimisé que le premier car il utilise des techniques plus avancées pour améliorer les performances.

université PARIS-SACLAY

TP3 : Computer vision

Algorithme 1 : kernel_seuillageGPU

Nous cherchons à réaliser un seuillage en temps réel de l'image de la caméra présente sur les cartes NVIDIA Jetson Nano en fonction de η_R, le niveau de rouge de chaque pixel de l'image de la caméra.

Pour cela, nous allons utiliser la formule suivante que nous avons déjà vue dans une partie précédente à savoir la 1.1 Ferrari rouge :

$$\eta_R = \frac{I_R}{\sqrt{I_R^2 + I_G^2 + I_B^2}}$$

Figure 41 : Formules afin de seuiller en fonction du niveau de rouge

Notre objectif est donc de parcourir chaque pixel de notre l'image de la caméra, puis de calculer le niveau de rouge de ce pixel via la formule $\eta_R = \frac{I_R}{\sqrt{I_R^2+I_G^2+I_B^2}}$. En fonction de la valeur de ce résultat, nous avons deux cas possibles. Le premier cas est lorsque $\eta_R > 0.7$, le pixel de sortie aura une teinte différente de l'entrée en ce qui concerne le canal vert. En effet, le canal vert du pixel de sortie sera égal au canal bleu de l'entrée, tandis que les canaux rouge et bleu de sortie seront égaux à ceux de l'entrée. On peut représenter ce cas avec l'exemple suivant :

- Entrée : RGB(255, 150, 50)
- Seuillage : $\eta_R > 0.7$
- Sortie : RGB(255, 50, 50)

Dans le second cas, lorsque $\eta_R \leq 0.7$, les trois canaux de l'image d'entrée sont totalement équivalents à ceux de l'image de sortie. On peut représenter ce cas avec l'exemple suivant :

- Entrée : RGB(100, 200, 150)
- Seuillage : $\eta_R \leq 0.7$
- Sortie : RGB(100, 200, 150)

Ainsi nous obtenons ce code en langage CUDA qui permet de filtrer en fonction du niveau de rouge et modifie les pixels si le niveau de rouge est supérieur à 0.7 :

```
__global__ void kernel_seuillageGPU(unsigned char *d_image_in, unsigned char *d_image_out,int size_j)
{
  float Csum;
  int i, j, k, iFirst, jFirst;

  iFirst = blockIdx.x*BLOCK_SIZE; // num de block dans la grille de block
  jFirst = blockIdx.y*BLOCK_SIZE;

  i = iFirst + threadIdx.x;// recuperer l'identifiant d'un thread dans les blocs
  j = jFirst + threadIdx.y;

  float nr = 0;

  nr=d_image_in[2+j*3+i*3*size_j]/sqrtf(d_image_in[0+j*3+i*3*size_j]*d_image_in[0+j*3+i*3*size_j]+d_image_in[1+j*3+i*3*size_j]*d_

  if(nr > 0.7)
    d_image_out[1+j*3+i*3*size_j] = d_image_in[2+j*3+i*3*size_j];
  else
    d_image_out[1+j*3+i*3*size_j] = d_image_in[1+j*3+i*3*size_j];

  d_image_out[0+j*3+i*3*size_j] = d_image_in[0+j*3+i*3*size_j];
  d_image_out[2+j*3+i*3*size_j] = d_image_in[2+j*3+i*3*size_j];
}
```

Figure 42 : Code qui filtre en fonction du niveau de rouge et modifie les pixels si le niveau de rouge est supérieur à 0.7

Voici le détail des lignes importantes concernant ce code :

Voici une explication détaillée de chaque ligne :
- « **iFirst = blockIdx.x*BLOCK_SIZE;** ». Cette ligne calcule l'index de la première colonne (i) de chaque bloc de threads en multipliant l'index du bloc par la taille de chaque bloc de threads (**BLOCK_SIZE**). Idem pour la première ligne (j).
- « **i = iFirst + threadIdx.x;** ». Cette ligne calcule l'index de la colonne (i) pour chaque thread en ajoutant « **iFirst** » défini précédemment à l'index du thread dans le bloc « **threadIdx.x** ». Idem pour la ligne (j).
- « **nr = d_image_in[2+j*3+i*3*size_j]/sqrtf(d_image_in[0+j*3+i*3*size_j]*d_image_in[0+j*3+i*3*size_j]+d_image_in[1+j*3+i*3*size_j]*d_image_in[1+j*3+i*3*size_j]+d_image_in[2+j*3+i*3*size_j]*d_image_in[2+j*3+i*3*size_j]);** ». Cette ligne calcule le niveau de rouge. Elle divise la valeur de la composante rouge par la racine carrée de la somme des carrés des valeurs de toutes les composantes (rouge, vert et bleu) pour le pixel à la position (i,j).
- « **if(nr > 0.7) d_image_out[1+j3+i3size_j] = d_image_in[2+j3+i3size_j];**
else d_image_out[1+j3+i3size_j] = d_image_in[1+j3+i3size_j]; ». Cette ligne applique le seuillage en comparant le niveau de rouge à une valeur seuil de 0.7. Si le ratio est supérieur à 0.7, la valeur de la composante verte (qui est le deuxième élément du tableau d_image_out) prend la valeur de la composante rouge (qui est le troisième élément du tableau d_image_in). Si le seuil est inférieur ou égal à 0.7 rien ne change, le pixel d'entrée est équivalent au pixel de sortie.

Puis vient la deuxième fonction qui est seuillageGPU et qui permet l'allocation mémoire, la copie des données du CPU vers le GPU et inversement ainsi que l'exécution du kernel « **kernel_seuillageGPU** » puis la libération de la mémoire.

```
Mat seuillageGPU( Mat in)
{
  cudaError_t error;
  Mat out;
  out.create(in.rows,in.cols,CV_8UC3);
  unsigned char *h_image_in_GPU ;
  h_image_in_GPU=in.data;

  unsigned char *d_image_in_GPU;
  unsigned char *d_image_out_GPU;

  const unsigned long int mem_size=in.cols*in.rows*3*sizeof(unsigned char);

  cudaMalloc((void**) &d_image_in_GPU,mem_size );
  cudaMalloc((void**) &d_image_out_GPU, mem_size);

  cudaMemcpy(d_image_in_GPU, h_image_in_GPU,mem_size ,cudaMemcpyHostToDevice);

  dim3 threads(BLOCK_SIZE,BLOCK_SIZE);
  dim3 grid(in.rows/BLOCK_SIZE,in.cols/BLOCK_SIZE);
  kernel_seuillageGPU<<< grid, threads >>>(d_image_in_GPU, d_image_out_GPU,in.cols);

  cudaMemcpy(out.data, d_image_out_GPU, mem_size,cudaMemcpyDeviceToHost);
  cudaFree(d_image_in_GPU);
  cudaFree(d_image_out_GPU);

  return out;
}
```

Figure 43 : Code permettant l'appel du kernel « kernel_seuillageGPU »

Ce code implémente une fonction CUDA pour effectuer un seuillage d'une image en utilisant la carte graphique. La fonction prend en entrée une image « **in** » de type « **Mat** » et renvoie une nouvelle image « **out** » également de type « **Mat** » où les pixels de l'image sont modifiés en fonction d'un seuil.

La première étape consiste à allouer de la mémoire pour stocker l'image d'entrée et l'image de sortie sur la carte graphique. Ensuite, la mémoire est copiée depuis le CPU vers le GPU en utilisant la fonction « cudaMemcpy ».

La fonction « kernel_seuillageGPU » vu précédemment est ensuite appelée en spécifiant les paramètres d'exécution. Le kernel utilise des threads pour itérer à travers chaque pixel de l'image et appliquer le seuillage.

Après l'exécution du kernel, les résultats sont copiés depuis le GPU vers le CPU avec la fonction « cudaMemcpy ». Enfin, la mémoire allouée sur le GPU est libérée en utilisant la fonction « cudaFree ». La fonction renvoie finalement l'image seuillée de sortie.

Algorithme 2 : kernel_AlgorithmeSobel

Notre objectif est de réaliser un code CUDA CUDA pour effectuer l'opérateur Sobel sur une image en niveaux de gris. L'algorithme Sobel est utilisé pour détecter les contours dans une image en calculant la dérivée de l'image dans les directions horizontale et verticale.

Afin de réaliser l'algorithme de Sobel, nous devons suivre quelques étapes principales à savoir :

- Charger l'image source en mémoire GPU et créer un tampon de sortie pour stocker les résultats
- Allouer de la mémoire sur le GPU
- Copier les données de la mémoire CPU vers la mémoire GPU
- Diviser l'image en blocs de threads. Pour chaque bloc, allouer une mémoire partagée pour stocker les sous-régions de l'image à traiter
- Lancer un kernel CUDA pour chaque bloc de threads. Chaque thread effectuera le calcul du gradient pour un pixel de l'image en utilisant les masques de Sobel :
  - Pour calculer le gradient, on utilise deux masques de convolution qui sont appliqués à l'image originale. Ces masques sont appelés masque de Sobel en X et en Y.
  - Pour calculer la composante X du gradient, on applique le masque de Sobel en X sur les pixels voisins horizontaux. Pour chaque pixel, on effectue la somme pondérée des valeurs des pixels voisins en multipliant chaque pixel par le coefficient de pondération correspondant du masque de Sobel en X. Cette somme pondérée est ensuite divisée par un facteur de normalisation, qui dépend de la taille du masque. On répète ce processus pour chaque pixel de l'image pour calculer la composante X du gradient. Nous faisons de même pour calculer la composante Y du gradient
  - Enfin, la magnitude du gradient est calculée pour chaque pixel via la formule suivante : $\text{Magnitude} = \sqrt{{g_x}^2 + {g_y}^2}$ en utilisant les composantes X et Y du gradient calculées précédemment. La magnitude du gradient représente la force et la direction du changement d'intensité des pixels dans l'image.
- Copier les résultats de la mémoire GPU vers la mémoire CPU
- Libérer la mémoire allouée sur le GPU

Au niveau de la programmation et manière détaillée, voici toutes les étapes qu'il a fallu suivre afin de refaire l'algorithme de Sobel :
Voici les étapes principales pour refaire le code :
- Déclarer deux tableaux d'entiers « **sobel_x** » et « **sobel_y** » qui sont nos deux masques de Sobel en X et en Y
- Calculer les indices « **idx** » et « **idy** » à partir des blocs et des threads, en utilisant « **threadIdx.x** », « **blockIdx.x** », « **threadIdx.y** », « **blockIdx.y** » et « **blockDim.x** »
- Calculer les positions des canaux de couleur rouge, vert et bleu à partir de « **posr** », « **posg** » et « **posb** »
- Vérifier que « **idx** » et « **idy** » se trouvent à l'intérieur des limites de l'image
- Calculer les valeurs de « **sobel.x** », « **sobel.y** » et « **sobel.z** » pour le gradient en Y à partir des positions des canaux de couleur en utilisant les valeurs de « **masqueSobelY** ». Faire de même pour le gradient en X
- Synchroniser les threads en utilisant « **__syncthreads()** »
- Calculer les valeurs de « **sobel.x** », « **sobel.y** » et « **sobel.z** » pour le gradient Y à partir des positions des canaux de couleur en utilisant les valeurs de « **masqueSobelY** »
- Calculer les valeurs du gradient de magnitude pour chaque canal de couleur

```cuda
int sobel_x[] = {-1, 0, 1};
int sobel_y[] = { 1, 2, 1};

__global__ void kernel_sobelGPU(unsigned char *d_image_in, unsigned char *d_buffer_gx, unsigned char *d_buffer_gy, unsigned char *d_image_out, int w, int h)
{
  int idx = threadIdx.x + blockIdx.x*blockDim.x;
  int idy = threadIdx.y + blockIdx.y*blockDim.y;
  const int posr = 3*(idx + w * idy), posg = posr + 1, posb = posg + 1;
  const int pas = 3;
  const int image_width = pas * w;
  int3 sobel;
  if(idx > 0 && idx < w-1 && idy > 0 && idy < h-1)
  {
    sobel.x = ((int)d_image_in[posr - pas] + (int)d_image_in[posr + pas] + 2*(int)d_image_in[posr]);
    sobel.y = ((int)d_image_in[posg - pas] + (int)d_image_in[posg + pas] + 2*(int)d_image_in[posg]);
    sobel.z = ((int)d_image_in[posb - pas] + (int)d_image_in[posb + pas] + 2*(int)d_image_in[posb]);

    d_buffer_gy[posr] = (unsigned char)sobel.x;
    d_buffer_gy[posg] = (unsigned char)sobel.y;
    d_buffer_gy[posb] = (unsigned char)sobel.z;

    __syncthreads();

    sobel.x = (int)d_buffer_gy[posr + image_width] - (int)d_buffer_gy[posr - image_width];
    sobel.y = (int)d_buffer_gy[posg + image_width] - (int)d_buffer_gy[posg - image_width];
    sobel.z = (int)d_buffer_gy[posb + image_width] - (int)d_buffer_gy[posb - image_width];

    d_buffer_gy[posr] = (unsigned char)sobel.x;
    d_buffer_gy[posg] = (unsigned char)sobel.y;
    d_buffer_gy[posb] = (unsigned char)sobel.z;
    sobel.x = (int)d_image_in[posr + pas] - (int)d_image_in[posr - pas];
    sobel.y = (int)d_image_in[posg + pas] - (int)d_image_in[posg - pas];
    sobel.z = (int)d_image_in[posb + pas] - (int)d_image_in[posb - pas];

    d_buffer_gx[posr] = (unsigned char)sobel.x;
    d_buffer_gx[posg] = (unsigned char)sobel.y;
    d_buffer_gx[posb] = (unsigned char)sobel.z;

    __syncthreads();

    sobel.x = (int)d_buffer_gx[posr - image_width] + (int)d_buffer_gx[posr + image_width] + 2*(int)d_buffer_gx[posr];
    sobel.y = (int)d_buffer_gx[posg - image_width] + (int)d_buffer_gx[posg + image_width] + 2*(int)d_buffer_gx[posg];
    sobel.z = (int)d_buffer_gx[posb - image_width] + (int)d_buffer_gx[posb + image_width] + 2*(int)d_buffer_gx[posb];

    d_buffer_gx[posr] = (unsigned char)sobel.x;
    d_buffer_gx[posg] = (unsigned char)sobel.y;
    d_buffer_gx[posb] = (unsigned char)sobel.z;

    __syncthreads();

    d_image_out[posr] = (unsigned char)sqrt( ((float) d_buffer_gx[posr] * (float) d_buffer_gx[posr]) + ( (float) d_buffer_gy[posr] * (float) d_buffer_gy[posr]) );
    d_image_out[posg] = (unsigned char)sqrt( ((float) d_buffer_gx[posg] * (float) d_buffer_gx[posg]) + ( (float) d_buffer_gy[posg] * (float) d_buffer_gy[posg]) );
    d_image_out[posb] = (unsigned char)sqrt( ((float) d_buffer_gx[posb] * (float) d_buffer_gx[posb]) + ( (float) d_buffer_gy[posb] * (float) d_buffer_gy[posb]) );
  }
}
```

Figure 44 : Code CUDA de l'algorithme Sobel

Pour résumé, le code définit deux tableaux d'entiers pour les opérateurs de Sobel dans les directions X et Y. La fonction principale est le kernel CUDA « kernel_sobelGPU » qui prend en entrée un pointeur vers une image en niveaux de gris d'entrée, deux tampons intermédiaires pour stocker les résultats de la détection de bord en X et en y, un pointeur de sortie pour stocker l'image de sortie, ainsi que la largeur et la hauteur de l'image.

Dans le noyau, les indices des blocs et des threads sont calculés pour accéder aux pixels de l'image d'entrée et de sortie. Les pixels de l'image sont lus en utilisant leurs indices de position dans les tableaux rouge, vert et bleu correspondants. Ensuite, le calcul de la détection de bord de Sobel est effectué en deux étapes pour les directions X et Y, en utilisant les tampons intermédiaires pour stocker les résultats partiels.

Les résultats finaux pour chaque pixel de couleur sont stockés dans l'image de sortie en prenant la racine carrée de la somme des carrés des gradients en X et Y pour chaque canal de couleur.